

Firebase for Android

Backend-as-a-Service for Mobile Development

HAI811I – Programmation Mobile

University of Montpellier – M1 – 2025-2026

Outline

- Introduction to Firebase
- Project Setup & Gradle Configuration
- Authentication
- Cloud Firestore
- Firebase Storage
- Cloud Messaging (FCM)
- Analytics & Crashlytics
- Security Rules
- Architecture Patterns
- Pricing, Limitations & Vendor Lock-in

1. Introduction to Firebase

What is Firebase and why use it?

What is Firebase?

- Comprehensive suite of tools and services by Google
- Backend-as-a-Service (BaaS): ready-to-use backend without managing servers
- Designed for mobile and web applications
- Real-time data handling, scalability, easy integration
- Covers: databases, authentication, storage, notifications, analytics, and more



Key Firebase Services

- Authentication: multiple sign-in methods
- Cloud Firestore: scalable NoSQL database
- Realtime Database: JSON-based real-time sync
- Cloud Storage: file uploads/downloads
- Cloud Functions: serverless backend logic
- Cloud Messaging (FCM): push notifications
- Analytics: user behavior tracking
- Crashlytics: crash reporting
- Remote Config: feature flags
- Hosting: static web hosting

Real-world Use Cases

- User authentication in social media and e-commerce apps
- Real-time chat applications using Firestore for message storage
- Photo/video sharing apps with Firebase Storage
- Push notifications for user engagement via FCM
- Analytics to understand user behavior and improve UX
- Crash monitoring with Crashlytics for production apps

Challenges and Limitations

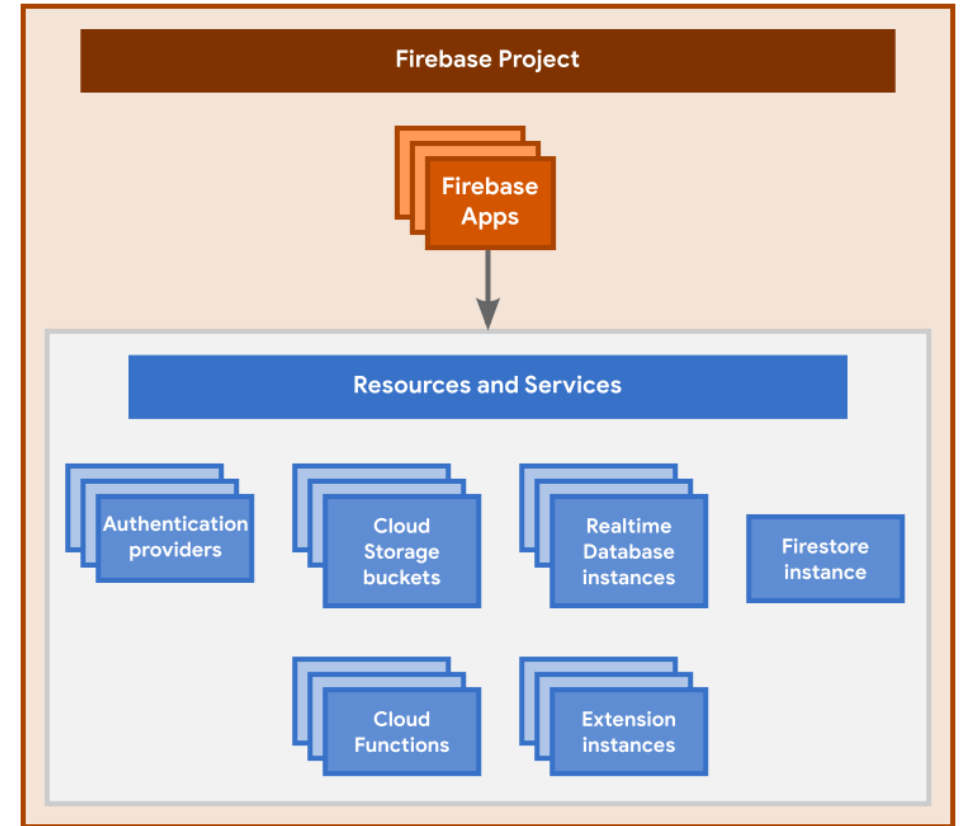
- Not suited for complex relational queries (NoSQL limitations)
- Vendor lock-in: deep dependency on Google ecosystem
- Limited control over backend logic compared to custom servers
- Free tier has limits; pay-as-you-go can become costly at scale
- Data sovereignty concerns for some organizations
- Offline support varies by service

2. Project Setup

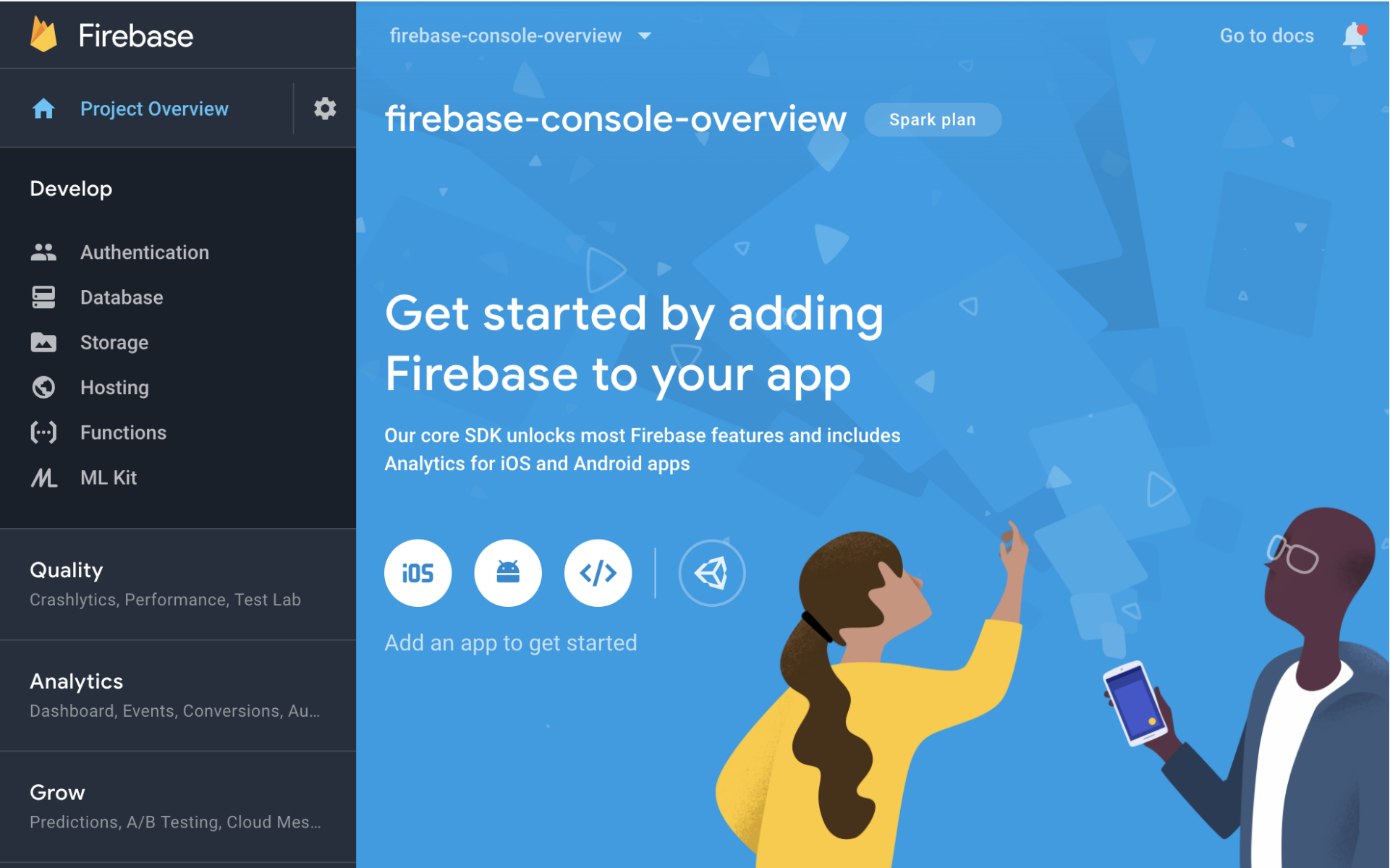
Firebase + Android Studio + Gradle

Creating a Firebase Project

- Go to firebase.google.com, sign in with Google
- Click 'Add project', enter a project name
- Optionally enable Google Analytics
- Project dashboard opens when done
- Click the Android icon to register your app
- Enter your app's package name (e.g., `com.example.myapp`)



Firebase Console Overview



The google-services.json File

- Generated when you register an Android app
- Contains project ID, API keys, app ID
- Place it in app/ directory of your Android project
- Do NOT commit to public repositories

```
// Key fields in google-services.json
{
  "project_info": {
    "project_id": "your-project-id",
    "storage_bucket": "...appspot.com"
  },
  "client": [{
    "client_info": {
      "mobilesdk_app_id": "1:123...:android:...",
      "android_client_info": {
        "package_name": "com.example.myapp"
      }
    },
    "api_key": [{ "current_key": "AIza..." }]
  }]
}
```

Gradle Configuration

```
// project-level build.gradle (or
settings.gradle)
plugins {
    id 'com.google.gms.google-services'
        version '4.4.2' apply false
}

// app-level build.gradle
plugins {
    id 'com.android.application'
    id 'com.google.gms.google-services'
}

dependencies {
    // Firebase BoM (Bill of Materials)
    implementation platform(
        'com.google.firebase:firebase-
bom:33.7.0')
    // Add Firebase SDKs without versions
    implementation
'com.google.firebase:firebase-analytics'
}
```

- Use Firebase BoM to manage versions
- BoM ensures all Firebase libraries are compatible
- Add only the SDKs you need
- Sync Gradle after changes
- google-services plugin processes the JSON config file

Firestore Initialization in Android

```
// MainActivity.java
import com.google.firebase.FirebaseApp;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Firebase auto-initializes with google-services
        plugin

        // Manual init only if needed:
        // FirebaseApp.initializeApp(this);

        // Verify connection
        if (FirebaseApp.getInstance() != null) {
            Log.d("Firestore", "Firestore is connected!");
        }
    }
}
```

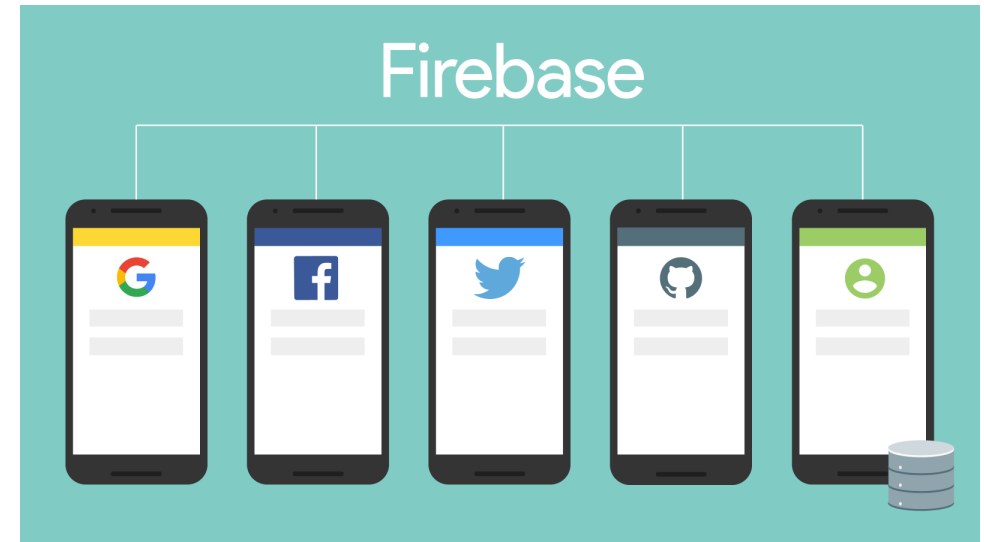
- google-services plugin auto-initializes Firestore
- No explicit init call needed in most cases
- Use `FirebaseApp.getInstance()` to verify connection
- Works with Activities, Fragments, Services

3. Firebase Authentication

Identity management for your app

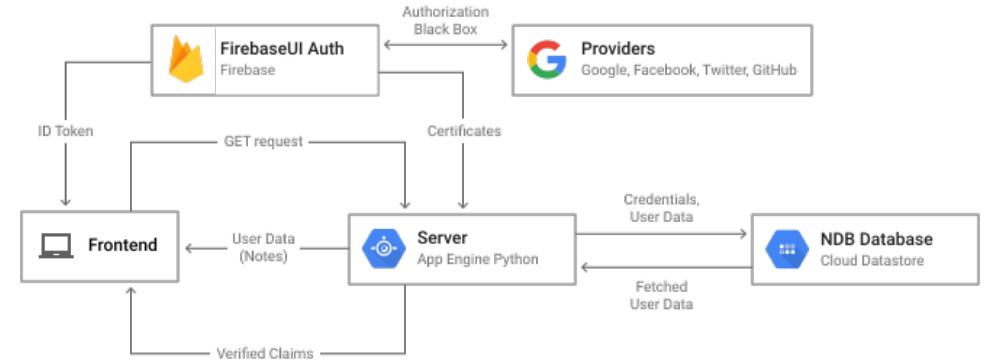
Authentication Overview

- Comprehensive identity solution by Firebase
- Supports email/password, phone, Google, Facebook, Twitter, GitHub, etc.
- Manages user sessions automatically
- Provides JWT tokens for secure API calls
- Integrates with Firestore security rules
- Free for most authentication methods



Authentication Setup

- In Firebase Console: Authentication > Sign-in method
- Enable desired providers (Email/Password, Google...)
- For Google Sign-In: configure OAuth consent screen



```
// app-level build.gradle - add Firebase Auth dependency
dependencies {
    implementation platform('com.google.firebase:firebase-bom:33.7.0')
    implementation 'com.google.firebase:firebase-auth'
}
```

Sign In with Email & Password

```
// AuthService.java
public class AuthService {
    private final FirebaseAuth mAuth =
        FirebaseAuth.getInstance();

    public void signIn(String email, String
        password, AuthCallback callback) {

        mAuth.signInWithEmailAndPassword(email,
            password)
            .addOnCompleteListener(task -> {
                if (task.isSuccessful()) {
                    FirebaseUser user =
                        mAuth.getCurrentUser();
                    callback.onSuccess(user);
                } else {
                    callback.onFailure(task.getException());
                }
            });
    }
}
```

```
public void register(String email, String
    password, AuthCallback callback) {

    mAuth.createUserWithEmailAndPassword(email,
        password)
        .addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                callback.onSuccess(mAuth.getCurrentUser());
            } else {
                callback.onFailure(task.getException());
            }
        });

    public void signOut() { mAuth.signOut(); }
}
```

Managing User Sessions

- Firebase persists auth state across app restarts
- Use AuthStateListener to react to sign-in/out
- Update UI based on authentication state

```
// In your Activity or ViewModel
FirebaseAuth.getInstance().addAuthStateListener(auth -> {
    FirebaseUser user =
auth.getCurrentUser();
    if (user != null) {
        // User is signed in - show main
content
        Log.d("Auth", "Signed in: " +
user.getEmail());
        navigateToHome();
    } else {
        // User is signed out - show login
screen
        navigateToLogin();
    }
});
```

Login Activity Example

```
public class LoginActivity extends AppCompatActivity
{
    private FirebaseAuth mAuth;
    private EditText emailField, passwordField;

    @Override
    protected void onCreate(Bundle
savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);
        mAuth = FirebaseAuth.getInstance();
        emailField = findViewById(R.id.email);
        passwordField = findViewById(R.id.password);

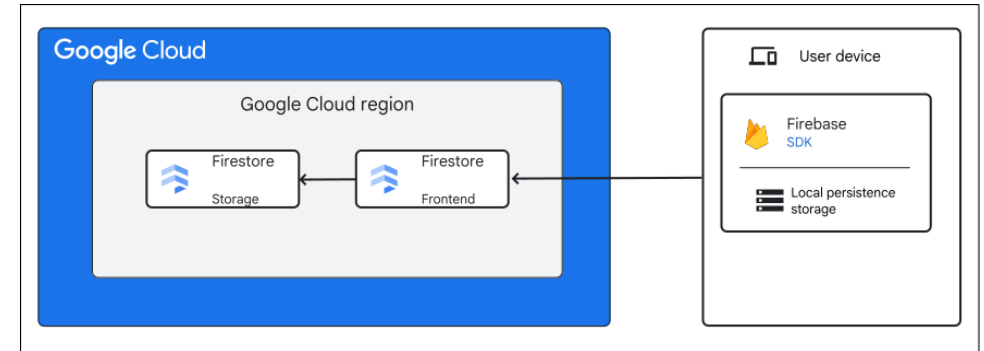
        findViewById(R.id.loginBtn).setOnClickListener(v ->
        {
            String email =
emailField.getText().toString();
            String password =
passwordField.getText().toString();
            mAuth.signInWithEmailAndPassword(email,
password)
                .addOnCompleteListener(this, task ->
                {
                    if (task.isSuccessful()) {
                        startActivity(new
Intent(this, HomeActivity.class));
                        finish();
                    } else {
                        Toast.makeText(this, "Auth
failed", Toast.LENGTH_SHORT).show();
                    }
                });
        });
    }
}
```

4. Cloud Firestore

Flexible, scalable NoSQL database

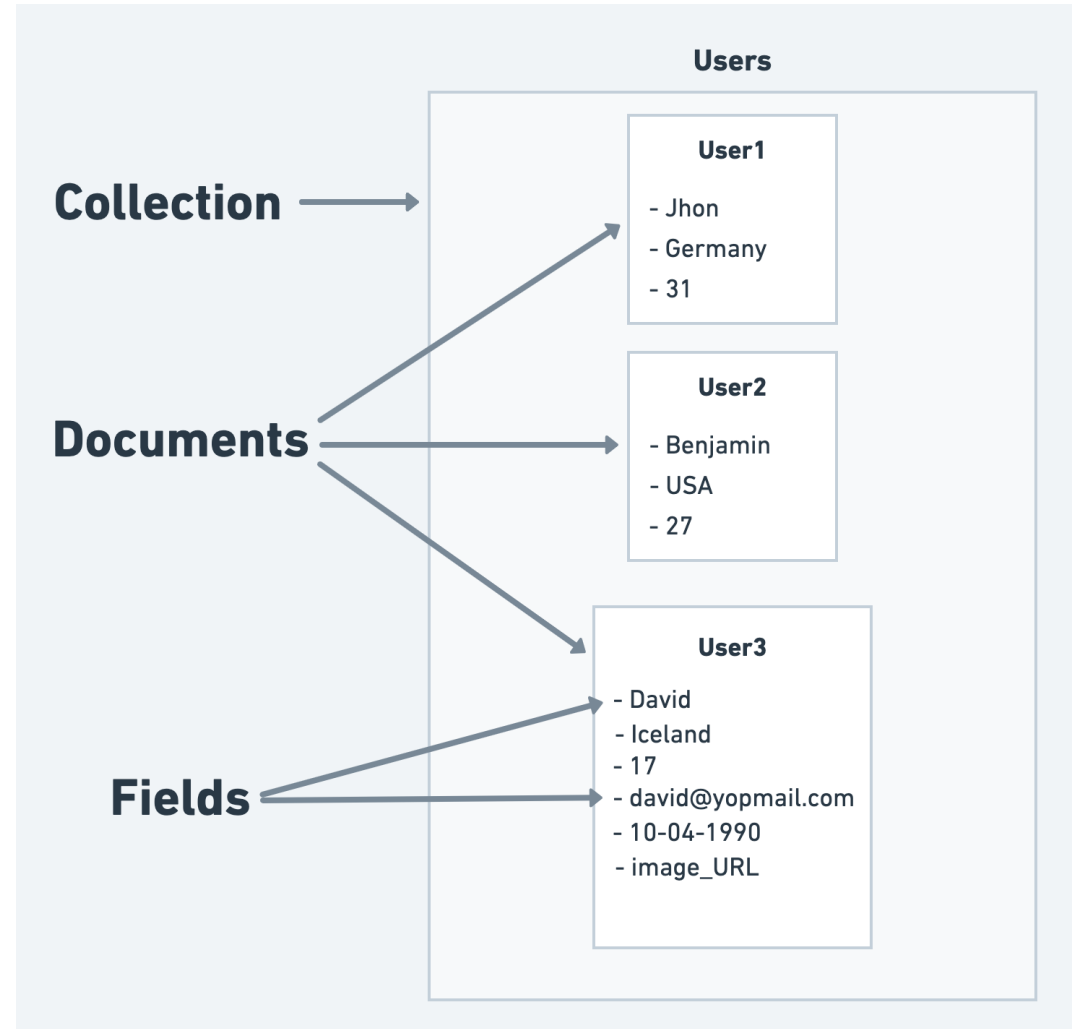
What is Cloud Firestore?

- NoSQL document database by Google
- Real-time data synchronization across clients
- Offline support with local caching
- Automatic scaling for large datasets
- Expressive querying capabilities
- Data organized in collections and documents



Firestore Data Model

- Collections: groups of documents
- Documents: key-value pairs (like JSON)
- Documents can contain subcollections
- Each document has a unique ID
- Supports: strings, numbers, booleans, arrays, maps, timestamps, geopoints
- Max document size: 1 MB



Firestore Setup

- Firebase Console > Firestore Database > Create
- Choose 'test mode' for development
- Select a database location (region)

```
// app-level build.gradle
dependencies {
    implementation platform('com.google.firebase:firebase-bom:33.7.0')
    implementation 'com.google.firebase:firebase-firestore'
}

// Get Firestore instance
FirebaseFirestore db = FirebaseFirestore.getInstance();
```

Firestore CRUD: Create & Read

```
// CREATE: Add a new document
Map<String, Object> user = new HashMap<>();
user.put("full_name", "John Doe");
user.put("email", "john@example.com");

db.collection("users")
    .add(user)
    .addOnSuccessListener(docRef ->
        Log.d("DB", "Added: " +
docRef.getId()))
    .addOnFailureListener(e ->
        Log.w("DB", "Error", e));
```

```
// READ: Get a document by ID
db.collection("users")
    .document(userId)
    .get()
    .addOnSuccessListener(doc -> {
        if (doc.exists()) {
            String name =
doc.getString("full_name");
            String email =
doc.getString("email");
            Log.d("DB", name + " - " +
email);
        }
    });

// READ: Get all documents in collection
db.collection("users")
    .get()
    .addOnSuccessListener(querySnapshot -> {
        for (DocumentSnapshot doc :
querySnapshot) {
            Log.d("DB", doc.getId());
        }
    });
```

Firestore CRUD: Update & Delete

```
// UPDATE: Modify a document
db.collection("users")
    .document(userId)
    .update("email", "newemail@example.com")
    .addOnSuccessListener(aVoid ->
        Log.d("DB", "Updated!"))
    .addOnFailureListener(e ->
        Log.w("DB", "Error", e));
```

```
// UPDATE: Multiple fields
Map<String, Object> updates = new
HashMap<>();
updates.put("email", "new@example.com");
updates.put("age", 25);
db.collection("users")
    .document(userId)
    .update(updates);
```

```
// DELETE: Remove a document
db.collection("users")
    .document(userId)
    .delete()
    .addOnSuccessListener(aVoid ->
        Log.d("DB", "Deleted!"))
    .addOnFailureListener(e ->
        Log.w("DB", "Error", e));
```

```
// DELETE: Remove a specific field
Map<String, Object> updates = new
HashMap<>();
updates.put("email", FieldValue.delete());
db.collection("users")
    .document(userId)
    .update(updates);
```

Real-time Data with Snapshot Listeners

```
// Listen to real-time updates on a collection
db.collection("users")
    .addSnapshotListener((querySnapshot, error) -> {
        if (error != null) { Log.w("DB", "Listen failed", error); return; }
        for (DocumentChange dc : querySnapshot.getDocumentChanges()) {
            switch (dc.getType()) {
                case ADDED:      Log.d("DB", "New: " + dc.getDocument().getData()); break;
                case MODIFIED:   Log.d("DB", "Modified: " + dc.getDocument().getData()); break;
                case REMOVED:    Log.d("DB", "Removed: " + dc.getDocument().getData()); break;
            }
        }
    });
```

- Listeners fire immediately with current data, then on every change
- Use DocumentChange.Type to handle ADDED, MODIFIED, REMOVED events
- Call remove() on the ListenerRegistration to stop listening

Firestore + RecyclerView Pattern

```
// In your Activity or Fragment - bind Firestore to RecyclerView
public class NotesActivity extends AppCompatActivity {
    private List<Note> noteList = new ArrayList<>();
    private NoteAdapter adapter;

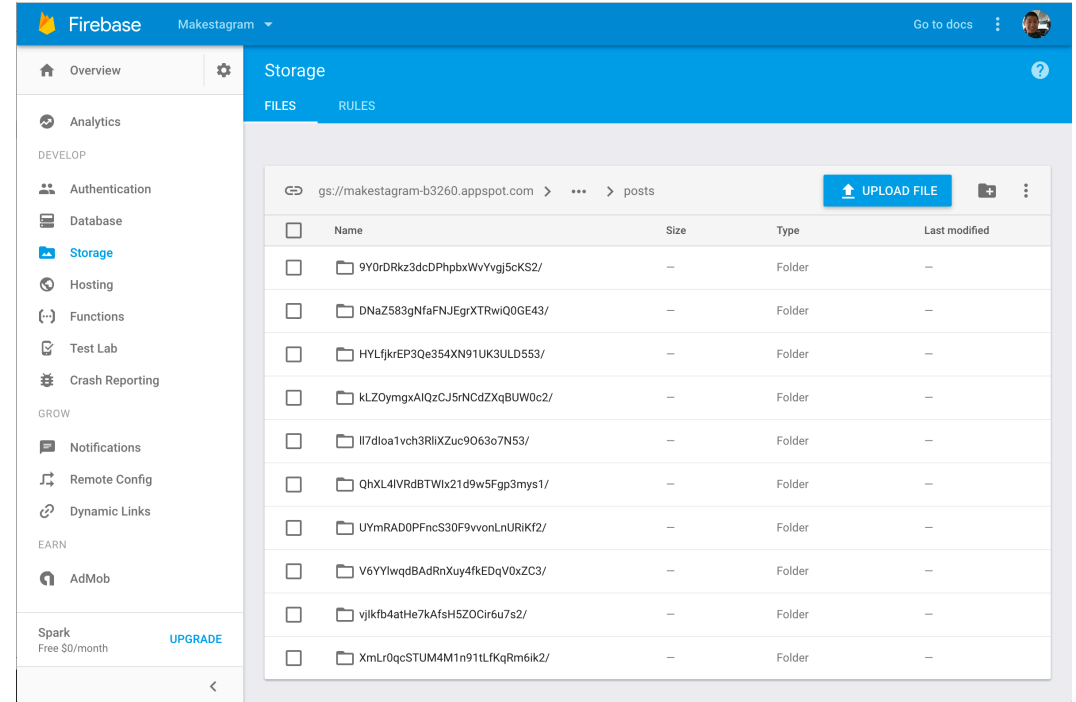
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_notes);
        RecyclerView recyclerView = findViewById(R.id.recyclerView);
        adapter = new NoteAdapter(noteList);
        recyclerView.setAdapter(adapter);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        // Real-time listener
        FirebaseFirestore.getInstance().collection("notes")
            .addSnapshotListener((snapshots, e) -> {
                noteList.clear();
                for (DocumentSnapshot doc : snapshots.getDocuments()) {
                    noteList.add(new Note(doc.getId(), doc.getString("content")));
                }
                adapter.notifyDataSetChanged();
            });
    }
}
```

5. Firebase Storage

File uploads and downloads at scale

Firestore Overview

- Built on Google Cloud Storage
- Upload/download files (photos, videos, docs)
- Handles poor network conditions gracefully
- Resumable uploads and downloads
- Integrates with Firebase Auth for security
- Accessible via storage references (paths)



Storage Setup & Security Rules

```
// app-level build.gradle
dependencies {
    implementation platform(
        'com.google.firebase:firebase-
bom:33.7.0')
    implementation
    'com.google.firebase:firebase-storage'
}
```

```
// Storage security rules
rules_version = '2';
service firebase.storage {
    match /b/{bucket}/o {
        match /{allPaths=**} {
            allow read, write: if
request.auth != null;
        }
    }
}
```

- Enable Storage in Firebase Console
- Configure security rules based on needs
- Default rules require authentication
- Use test mode for development only
- Secure before production deployment
- Rules support path-based access control

Uploading Files to Firebase Storage

```
// Upload a file to Firebase Storage
public void uploadFile(Uri fileUri) {
    FirebaseStorage storage = FirebaseStorage.getInstance();
    StorageReference ref = storage.getReference()
        .child("uploads/" + fileUri.getLastPathSegment());

    ref.putFile(fileUri)
        .addOnProgressListener(snapshot -> {
            double progress = (100.0 * snapshot.getBytesTransferred())
                / snapshot.getTotalByteCount();
            Log.d("Upload", "Progress: " + progress + "%");
        })
        .addOnSuccessListener(taskSnapshot -> {
            // Get download URL after successful upload
            ref.getDownloadUrl().addOnSuccessListener(uri -> {
                String downloadUrl = uri.toString();
                Log.d("Upload", "URL: " + downloadUrl);
            });
        })
        .addOnFailureListener(e ->
            Log.e("Upload", "Failed", e));
}
```

Downloading & Displaying Files

```
// Get download URL
StorageReference ref = FirebaseStorage
    .getInstance().getReference()
    .child("uploads/photo.jpg");

ref.getDownloadUrl()
    .addOnSuccessListener(uri -> {
        // Use Glide to display the image
        Glide.with(context)
            .load(uri)
            .into(imageView);
    });
```

```
// Delete a file
StorageReference ref = FirebaseStorage
    .getInstance().getReference()
    .child("uploads/photo.jpg");

ref.delete()
    .addOnSuccessListener(aVoid -> {
        Log.d("Storage", "File deleted");
    })
    .addOnFailureListener(e -> {
        Log.e("Storage", "Delete failed", e);
    });

// List all files in a directory
StorageReference listRef = storage
    .getReference().child("uploads/");
listRef.listAll()
    .addOnSuccessListener(listResult -> {
        for (StorageReference item :
            listResult.getItems()) {
            Log.d("Storage", item.getName());
        }
    });
```

Android Image Picker + Upload Pattern

```
// Using ActivityResultLauncher (modern approach)
private ActivityResultLauncher<Intent> pickImageLauncher;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_gallery);

    pickImageLauncher = registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(), result -> {
            if (result.getResultCode() == RESULT_OK && result.getData() != null) {
                Uri imageUri = result.getData().getData();
                uploadFile(imageUri); // Upload to Firebase Storage
            }
        });

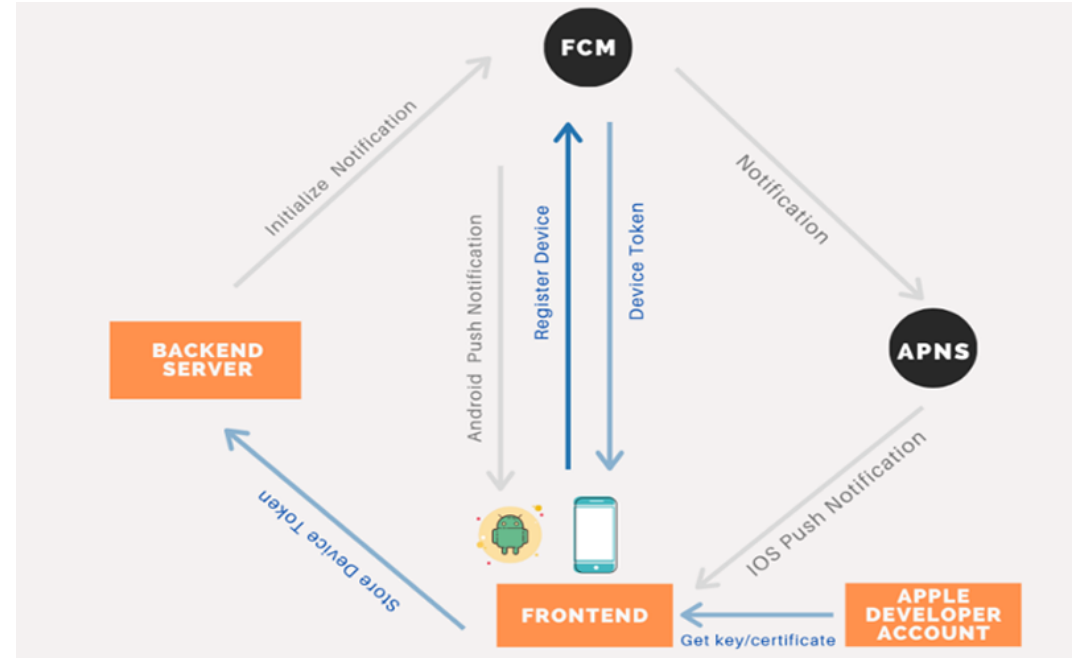
    findViewById(R.id.btnPickImage).setOnClickListener(v -> {
        Intent intent = new Intent(Intent.ACTION_PICK,
            MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
        pickImageLauncher.launch(intent);
    });
}
```

6. Firebase Cloud Messaging

Push notifications across platforms

FCM Overview

- Send notifications and data messages to devices
- Works on Android, iOS, and Web
- Key use cases:
 - Push notifications for user engagement
 - Real-time updates and alerts
 - Targeted messaging by topic or user segment
- Free, no limit on number of messages
- Two message types: notification & data



FCM Setup for Android

```
// app-level build.gradle
dependencies {
    implementation platform(
        'com.google.firebase:firebase-
bom:33.7.0')
    implementation
        'com.google.firebase:firebase-
messaging'
}

// AndroidManifest.xml
<service
    android:name=".MyFirebaseMessagingService"
    android:exported="false">
    <intent-filter>
        <action android:name=
"com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

- Add firebase-messaging dependency
- Register service in AndroidManifest.xml
- Extend FirebaseMessagingService
- Handle token refresh for device targeting
- No explicit permission needed on Android (unlike iOS)

FCM Service Implementation

```
public class MyFirebaseMessagingService
extends FirebaseMessagingService {

    @Override
    public void onNewToken(@NonNull String
token) {
        super.onNewToken(token);
        Log.d("FCM", "New token: " + token);
        // Send token to your server for
        targeted messaging
        sendTokenToServer(token);
    }

    @Override
    public void onMessageReceived(@NonNull
RemoteMessage remoteMessage) {
        super.onMessageReceived(remoteMessage);

        // Handle notification payload
        if (remoteMessage.getNotification() !=
null) {
            String title =
remoteMessage.getNotification().getTitle();
            String body =
remoteMessage.getNotification().getBody();
            showNotification(title, body);
        }

        // Handle data payload
        if (remoteMessage.getData().size() > 0) {
            Map<String, String> data =
remoteMessage.getData();
            handleDataMessage(data);
        }
    }
}
```

Displaying Notifications

```
private void showNotification(String title, String body) {
    String channelId = "default_channel";
    // Create notification channel (required for Android 8+)
    NotificationChannel channel = new NotificationChannel(
        channelId, "Default", NotificationManager.IMPORTANCE_HIGH);
    NotificationManager manager = getSystemService(NotificationManager.class);
    manager.createNotificationChannel(channel);

    // Build the notification
    Intent intent = new Intent(this, MainActivity.class);
    intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        this, 0, intent, PendingIntent.FLAG_IMMUTABLE);
    NotificationCompat.Builder builder = new NotificationCompat.Builder(this, channelId)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle(title)
        .setContentText(body)
        .setAutoCancel(true)
        .setContentIntent(pendingIntent);

    manager.notify(0, builder.build());
}
```

7. Analytics & Crashlytics

Understand users and fix crashes

Firebase Analytics

- Automatically collects: app_open, session_start, user_engagement, screen views
- Log custom events for specific interactions
- Set user properties for segmentation
- View reports in Firebase Console
- Integrates with BigQuery for advanced analysis
- Free and unlimited



Logging Analytics Events

```
// Get Analytics instance
FirebaseAnalytics analytics =
    FirebaseAnalytics.getInstance(this);

// Log a custom event
Bundle params = new Bundle();
params.putString("item_name", "Premium
Plan");
params.putDouble("price", 9.99);
analytics.logEvent("purchase", params);

// Log a predefined event
Bundle viewParams = new Bundle();
viewParams.putString(
    FirebaseAnalytics.Param.SCREEN_NAME,
    "HomeScreen");
analytics.logEvent(
    FirebaseAnalytics.Event.SCREEN_VIEW,
    viewParams);
```

```
// Set user properties
analytics.setUserProperty(
    "preferred_language", "Java");

analytics.setUserProperty(
    "subscription_type", "premium");

// Set user ID for cross-device tracking
analytics.setUserId("user_12345");

// Log event with no parameters
analytics.logEvent(
    "tutorial_completed", null);

// Log event for button click
Bundle clickParams = new Bundle();
clickParams.putString("button_id",
    "submit_form");
analytics.logEvent(
    "button_click", clickParams);
```

Firebase Crashlytics

- Lightweight crash reporter for production apps
- Automatic crash reporting with stack traces
- Custom log messages and keys for context
- Prioritizes crashes by impact on users

```
// build.gradle: id 'com.google.firebase.crashlytics' (app-level plugin)
// Log custom messages
FirebaseCrashlytics.getInstance().log("User clicked checkout");

// Set custom keys for crash context
FirebaseCrashlytics.getInstance().setCustomKey("user_plan", "premium");

// Record non-fatal exceptions
try { riskyOperation(); }
catch (Exception e) { FirebaseCrashlytics.getInstance().recordException(e); }
```

8. Security Rules

Protecting your data

Firebase Security Rules

- Declarative rules that control access to Firestore and Storage
- Evaluated server-side (cannot be bypassed by client)
- Based on authentication state, request data, existing data

```
// Firestore rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users can only read/write their own
    data
    match /users/{userId} {
      allow read, write:
        if request.auth != null
          && request.auth.uid == userId;
    }
  }
}
```

```
// Storage rules
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    // Only authenticated users can upload
    // Max file size: 5 MB
    match /uploads/{fileName} {
      allow read: if request.auth != null;
      allow write: if request.auth != null
        && request.resource.size < 5 * 1024
          * 1024;
    }
  }
}
```

Security Rules Best Practices

- Never use open rules (allow read, write: if true) in production
- Always validate data structure in rules (e.g., required fields, types)
- Use `request.auth` to restrict access to authenticated users
- Scope access to user-owned data using `request.auth.uid`
- Test rules with the Firebase Emulator Suite before deploying
- Use Firebase Console Rules Playground for quick testing
- Keep rules as simple as possible

9. Architecture Patterns

Integrating Firebase in Android apps

MVVM Architecture with Firebase

- Model: data classes + Firebase services
- View: Activities/Fragments observe LiveData
- ViewModel: mediates between View and Model
- Repository pattern: abstracts data sources
- Firebase operations in Repository layer
- LiveData/Flow for reactive UI updates

Architecture Flow

=====

Activity / Fragment

|

| observes LiveData

v

ViewModel

|

| calls methods

v

Repository

/

\

/

\

Firestore
(remote)

Firebase Auth
(remote)

Repository encapsulates all Firebase interactions.

ViewModel exposes LiveData for the UI to observe.

Repository Pattern Example

```
public class NoteRepository {
    private final FirebaseFirestore db =
        FirebaseFirestore.getInstance();
    private final MutableLiveData<List<Note>>
        notesLiveData = new MutableLiveData<>();

    public LiveData<List<Note>> getNotes() {
        db.collection("notes").addSnapshotListener((snapshots, e) -> {
            if (e != null || snapshots == null)
                return;

            List<Note> notes = new ArrayList<>();
            for (DocumentSnapshot doc :
                snapshots.getDocuments()) {
                notes.add(new Note(doc.getId(),
                    doc.getString("content")));
            }
            notesLiveData.setValue(notes);
        });
        return notesLiveData;
    }
}
```

```
public void addNote(String content) {
    Map<String, Object> note = new
        HashMap<>();
    note.put("content", content);
    db.collection("notes").add(note);
}

public void deleteNote(String id) {
    db.collection("notes").document(id).delete();
}
}
```

ViewModel + LiveData Example

```
// NoteViewModel.java
public class NoteViewModel extends ViewModel
{
    private final NoteRepository repository;
    private final LiveData<List<Note>>
notes;

    public NoteViewModel() {
        repository = new NoteRepository();
        notes = repository.getNotes();
    }

    public LiveData<List<Note>> getNotes() {
        return notes;
    }

    public void addNote(String content) {
        repository.addNote(content);
    }

    public void deleteNote(String id) {
        repository.deleteNote(id);
    }
}
```

```
// In Activity or Fragment
public class NotesFragment extends Fragment {
    private NoteViewModel viewModel;

    @Override
    public void onCreateView(View view,
        Bundle savedInstanceState) {
        super.onCreateView(view,
savedInstanceState);
        viewModel = new ViewModelProvider(this)
            .get(NoteViewModel.class);
        viewModel.getNotes().observe(
            getViewLifecycleOwner(),
            notes -> {
                // Update RecyclerView adapter
                adapter.submitList(notes);
            });
        btnAdd.setOnClickListener(v -> {
            String text = editText.getText()
                .toString();
            viewModel.addNote(text);
        });
    }
}
```

10. Additional Firebase Services

Realtime Database, Remote Config, Cloud Functions

Realtime Database vs Cloud Firestore

▶ Realtime Database:

- ▶ - JSON tree structure
- ▶ - Low latency for simple data
- ▶ - Pricing: based on bandwidth + storage
- ▶ - Better for: real-time sync, presence
- ▶ - Limited querying capabilities

▶ Cloud Firestore:

- ▶ - Collections/Documents structure
- ▶ - Rich querying and indexing
- ▶ - Pricing: based on reads/writes/deletes
- ▶ - Better for: complex apps, scaling
- ▶ - Recommended for new projects

Firebase Remote Config

- Change app behavior without publishing updates
- Define key-value parameters in Firebase Console
- A/B testing and gradual feature rollouts

```
FirebaseRemoteConfig config = FirebaseRemoteConfig.getInstance();
config.setDefaultsAsync(R.xml.remote_config_defaults);

config.fetchAndActivate().addOnCompleteListener(this, task -> {
    if (task.isSuccessful()) {
        boolean showBanner = config.getBoolean("show_promo_banner");
        String welcomeMsg = config.getString("welcome_message");
        updateUI(showBanner, welcomeMsg);
    }
});
```

Firebase Cloud Functions

- Serverless backend code triggered by events
- Written in JavaScript/TypeScript (Node.js)
- Triggers: Firestore changes, Auth events, HTTP requests
- Use for: validation, aggregation, sending emails

```
// Example Cloud Function (Node.js) - runs server-side
exports.onUserCreated = functions.auth.user().onCreate((user) => {
  return admin.firestore().collection('users').doc(user.uid).set({
    email: user.email,
    createdAt: admin.firestore.FieldValue.serverTimestamp(),
  });
});
```

11. Tradeoffs & Pricing

Making informed decisions

Firebase Pricing Overview

▶ Spark Plan (Free):

- ▶ - 1 GB Firestore storage
- ▶ - 50K reads / 20K writes per day
- ▶ - 5 GB Storage, 1 GB/day download
- ▶ - 10K FCM messages/month
- ▶ - Analytics: unlimited

▶ Blaze Plan (Pay-as-you-go):

- ▶ - \$0.06/100K Firestore reads
- ▶ - \$0.18/100K Firestore writes
- ▶ - \$0.026/GB Storage per month
- ▶ - Cloud Functions: \$0.40/million invocations
- ▶ - Set budget alerts in Google Cloud Console

Vendor Lock-in Considerations

- Firebase is deeply tied to Google Cloud Platform
- Migrating away requires rewriting backend logic and data access
- Firestore has no direct equivalent outside Google (unlike SQL databases)
- Authentication can be partially replaced (but token formats differ)
- Mitigation: use Repository pattern to abstract Firebase behind interfaces
- Mitigation: avoid using too many Firebase-specific features unnecessarily
- Evaluate: is Firebase the right fit for your project's long-term needs?

Summary

Key Takeaways

What We Covered

- Firebase = BaaS by Google
- Project setup: Console + google-services.json + Gradle
- Authentication: email/password, Google, sessions
- Cloud Firestore: NoSQL, CRUD, real-time listeners
- Firebase Storage: upload, download, delete files
- Cloud Messaging: push notifications on Android
- Analytics: event tracking, user properties
- Crashlytics: crash reporting
- Security rules: server-side access control
- MVVM + Repository: clean architecture with Firebase

Best Practices Checklist

- Use Firebase BoM to manage dependency versions
- Never commit google-services.json to public repos
- Always secure rules before production deployment
- Use Repository pattern to decouple Firebase from your UI
- Monitor usage and set budget alerts on Blaze plan
- Use the Firebase Emulator Suite for local testing
- Prefer Firestore over Realtime Database for new projects
- Keep sensitive logic in Cloud Functions, not on the client

Questions?

firebase.google.com/docs